

Senior Software Engineer

Technical Vetting Assignment

Instructions & Requirements

Take Home Assignment: Loyalty Points Wallet (Go)

Context

Sanlam's SFTX division powers loyalty and financial products used across the group. Your team builds backend services that track member accounts, enforce business rules, and integrate with other systems. This exercise focuses on a minimal "points wallet" that supports accounts, earning/spending of points, simple access control, and safe handling of duplicate or concurrent submissions — problems you'll see in identity, loyalty, and financial services.

Requirements

Build a small backend service for a loyalty points wallet.

Core tech stack:

- Go
- A relational database with data persisted across restarts (SQLite recommended for local development; PostgreSQL also acceptable)

General notes:

- You decide the API design, endpoints, data model, and overall structure
- Keep it runnable locally
- Use plain language in your SOLUTION.md to explain your choices
- In SOLUTION.md, include a short section showing the prompts or AI workflows you used (what you asked, what you accepted/edited, and why)

Example data formats (you may adapt as needed):

- Account (JSON)
 - { "account_id": "member-123", "name": "Rina" }
- Transaction (JSON)
 - { "ref": "tx-001", "account_id": "member-123", "kind": "earn" | "spend", "points": 150, "occurred_at": "2024-06-01T10:00:00Z" }
- CSV for batch ingestion
 - Header: ref,account_id,kind,points,occurred_at
 - Example row: tx-001,member-123,earn,150,2024-06-01T10:00:00Z

Constraints to consider:

- Prevent counting the same transaction more than once when the same reference appears again
- Do not allow a spend that would drive the balance below zero
- Handle overlapping requests on the same account safely so balances remain correct
- Keep data durable across process restarts

Task 1: Points wallet service

Build a service that:

- Creates and manages member accounts
- Records earn and spend operations against an account
- Tracks current balance per account
- Prevents double counting when the same "ref" is submitted more than once

- Rejects spends that would make the balance negative
- Provide example requests/responses in your README for the endpoints or interfaces you design.

Task 2: Access control

Add a simple access model:

- At least two roles: member and admin
- Members can read their own balance and submit their own earn/spend
- Admins can view any account and apply adjustments

How you issue and verify access is up to you (for example, static tokens, signed tokens, or another approach). Document the shape of the token/credential you chose, how it's stored/validated, and how roles are enforced.

Task 3: Batch ingestion and concurrency safety

Add a way to ingest a list of transactions from a CSV file (via an endpoint or a CLI entry point — your choice). It should:

- Apply transactions safely even if rows for the same account appear close together or the file is reprocessed
- Produce a short summary once complete (for example: total rows processed, accepted, rejected, duplicates)
- Write a simple audit trail that records each attempt (accepted or rejected) with a reason and timestamp (file, console, or table — your choice)

Deliverables

1. A single GitHub repository containing:

- All the source code for the assignment
- A README.md with instructions to run locally.
- A SOLUTION.md describing the design and trade-offs you made.

2. Video Demo (5-10 minutes using Loom or similar):

- Demonstrate the working project by doing a voice over
- Walk through your code architecture
- Explain key design decisions and any trade-offs made
- Highlight interesting technical implementations

Time Estimate

This should take ± 4 hours to complete, but you can take as long as you need.